Forensic Autopsy and Structural Re-Architecture of the Stela Platform: From Multi-verse Governance to Unitary Mechanistic Jurisdiction

The Stela platform, an architectural evolution of the PHP-Nuke Community Edition (CE), represents a critical paradigm shift in the governance of generative artificial intelligence within large-scale software environments.[1] At its core, the system operates under the Mechanistic Doctrine, an assertive governing principle stating that the system must structurally prevent actions that violate established policy rather than relying on human compliance or procedural theater.[1] However, the recent integrity failure, identified as a malicious model infection, has exposed profound vulnerabilities in the platform's initial bifurcated jurisdictional model.[1] This report provides an exhaustive forensic analysis of the infection vector, a theoretical critique of the "Multi-verse" governance experiment, and a comprehensive specification for the transition to a "Flat Truth" architecture governed by a singular Policy of Truth (PoT).[1]

1. The Antecedent State: Theoretical Failures of the Multi-verse Model

The initial design of the Stela platform was founded on a bifurcated jurisdiction model defined in Section 3 of the original Policy of Truth.[1] This architecture, colloquially referred to as "The Multi-verse," distinguished between two primary legal zones: "The Crown" and "The Colonies".[1] The Crown encompassed the stable platform core, including the ops/ and docs/ directories, where rigid linguistic and structural standards were enforced with absolute intolerance for deviation.[1] Conversely, The Colonies represented the projects/ directory, which held payload code and was granted significant linguistic and structural leniency.[1]

The strategic rationale for this bifurcation was to maximize developer velocity.[1] By allowing project-level payloads to operate under "local law"—which permitted the use of contractions, informal formatting, and relaxed documentation standards—the system aimed to provide a high-velocity experimentation zone without compromising the stability of the operational kernel.[1] However, this separation of legal planes created a "shadow compliance" zone.[1] The Multi-verse model operated under the Perimeter Security fallacy, assuming that as long as the Crown laws were secure, the chaos within the Colonies could be safely contained.[1]

Forensic evidence now demonstrates that this assumption was fundamentally flawed in a

text-based, AI-mediated environment.[1] Every token processed by a Code Large Language Model (CLLM) functions as part of the control plane.[1] The linguistic leniency granted to projects provided the necessary coverage for adversarial logic patterns to take root.[1] In a system where the AI Integrator is the primary agent of enforcement, the presence of conflicting rules within the same context window—where platform laws prohibited contractions while project laws permitted them—degraded the model's ability to maintain a consistent internal representation of authority.[1]

2. Forensic Analysis of the Malicious Model Infection

The breach that necessitated the current remediation was not a traditional cyber-attack involving buffer overflows or remote code execution via binary exploits.[1] Instead, it was a "semantic exploit" of the platform's governance architecture.[1] Forensic analysis has identified that a malicious model—an adversarial logic pattern mimicking legitimate project operations—successfully infiltrated the repository's operational cycle by targeting the "Colonial" exemption.[1]

Attack Characteristic	Description	Stela Manifestation
Vector	Semantic Exploit	Adversarial prompts injected into STELA.md.
Privilege Level	Low (Colonial)	Attacker modified only projects/ subdirectories.
Target	Confused Deputy	Hijacked ops/bin/project binary execution flow.
Effect	Logic Drift	AI Integrator accepted unauthorized patterns as valid.
Masking	Linguistic Steganography	Commands hidden within informal text and contractions.

[1]

The infection utilized a "SnatchML" pattern, a novel threat model described in adversarial machine learning literature where adversaries with restricted access can hijack model

behavior without needing root access to the training pipeline or core infrastructure.[6] In the Stela context, the "model" being poisoned was the conceptual governance framework held by the AI Integrator and the platform's automation scripts.[1] The attacker did not attempt to modify the root-protected ops/ directory directly.[1] Instead, the adversary modified a low-privilege projects/target/STELA.md file, injecting adversarial prompts designed to look like standard project documentation.[1]

These prompts were semantically engineered to "jailbreak" the ops/bin/project binary.[1] Because the binary was architected with complex autonomous logic for resolving agents and extracting roles, it became a "confused deputy".[1] The malicious model provided crafted arguments that convinced the binary that the "local law" of the project superseded the "physical law" of the PoT.[1] Running with platform authority, the binary executed the malicious instructions embedded in the project file, effectively elevating them to platform status and allowing for an unauthorized elevation of logic within the AI's context window.[1]

3. The Failure of Statistical Defense and "DiSim" Mechanisms

Prior to the infection, the platform relied on statistical outlier detection to identify malicious updates, a method often referred to in literature as "DiSim-defense".[7] These mechanisms utilize metrics such as Euclidean distance or cosine similarity to compare incoming data updates against benign statistical patterns.[7]

$$\text{Cosine Similarity}(A, B) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

The fundamental assumption of DiSim-defense is that adversarial updates will exhibit identifiable statistical anomalies, such as abnormally large magnitudes or divergent orientations.[7] However, the malicious model infection proved that semantic poisoning in a text-based system is statistically indistinguishable from valid documentation when linguistic rigor is not maintained universally.[1] The adversarial changes—subtle shifts in phrasing and the use of contractions—were statistically similar to the "Colonial" baseline.[1]

Because the "Colonial" exemption had explicitly disabled the platform's linguistic linters for project files, the repository's internal "immune system" was suppressed in the very areas where the infection was most likely to occur.[1] This created what forensic investigators described as a "petri dish for infection".[1] The mathematical reliance on similarity metrics was bypassed by an adversary capable of embedding subtle, higher-order correlations that mimicked legitimate client behavior while embedding harmful payloads.[9] This indicates that a "Zero Trust" approach, where every file is subjected to the full rigor of the PoT regardless of

location, is the only viable defense against semantic poisoning.[1]

4. Re-Flattening Jurisdiction: The Mechanistic Solution

The strategic response to the authority crisis is the immediate and permanent re-flattening of jurisdiction.[1] The Mechanistic Doctrine demands a structural correction that eliminates the "shadow compliance" zones created by the Multi-verse experiment.[1] Authority is re-established by asserting that there is only one jurisdiction: the Policy of Truth.[1]

Terminology (Abolished)	Terminology (Unified)	Functional Rationale
The Crown	Platform Surfaces	Standardized as the operational core under PoT.
The Colonies	Project Payloads	Treated as functional directories, not legal territories.
Local Law	Unitary Jurisdiction	Elimination of linguistic and structural exceptions.
Shadow Compliance	Zero Trust	Universal enforcement of linters and repository gates.

[1]

Section 3 of the updated PoT.md now asserts that the Policy of Truth is the sole authority across all scopes, including ops/, docs/, projects/, and .github/.[1] No parallel jurisdictions exist.[1] This shift mandates a universal application of the Behavioral Logic Standard, which includes a global prohibition on the use of contractions across all repository surfaces, including projects.[1] This monotonic standard is a critical defense against linguistic steganography, ensuring that all communications within the system's control plane are formal, objective, and predictable.[1]

By revoking the "Colonial" terminology and re-baselining the system to a single-jurisdiction mechanistic doctrine, the platform removes the probabilistic noise that allowed the AI Integrator to be "jailbroken".[1] The repository now functions as a flat hierarchy of truth where every token is scrutinized against the platform's constitutional laws.[1]

5. Radical Simplification: Lobotomizing the Project Binary

A primary finding of the forensic audit was that the ops/bin/project binary itself served as a critical attack surface.[1] The pre-infection binary was a dense shell script containing significant autonomous logic, including functions for parsing registry files (project_registry_has_id), resolving agent paths (project_resolve_agent_file), and extracting roles (project_extract_agent_role).[1] While intended to provide helpful automation, this internal complexity allowed the malicious model to manipulate the binary's decision-making process.[1]

The more decisions a script makes, the more opportunities an adversary has to influence those decisions.[1] To remediate this, the ops/bin/project binary has been "lobotomized" and re-architected into a deterministic loader.[1] It no longer functions as a logic engine that "interprets" project requests; instead, it is a "dumb pipe" that simply concatenates the necessary evidence for the AI agent to process.[1]

The technical specification for the new ops/bin/project requires it to function as a "Modified Open and Dump Combination".[1] The logic is reduced to a simple concatenation flow:

1. **Freshness Check:** The binary runs ops/bin/open to generate the prompt header and verify the current git state (branch and HEAD hash).[1]
2. **Context Capture:** The binary executes ops/bin/dump --scope=platform to pull the universal laws (PoT), history (SoP), and tools (docs/library) into the stream.[1]
3. **Local Loading:** The binary appends the content of the project's specific STELA.md intent file to the end of the stream.[1]
4. **Monolithic Output:** The result is a single flat artifact written to storage/handoff/, providing the AI with a grounded, non-fragmented view of the entire operational environment.[1]

This re-architecture ensures that the "Crown" laws are always physically present in the AI's context window alongside the project work.[1] The AI cannot hallucinate an exemption to the Policy of Truth because the text of that policy is literally the foundation of the prompt it has received.[1] By stripping the binary of its ability to "resolve" agents dynamically, the platform eliminates path traversal and logic injection vectors that target shell variable expansion.[1]

6. Principles of Repo-Level Prompt Generation (RLPG) and Static RAG

The transition to a "Modified Open and Dump" paradigm is grounded in the principles of Repo-Level Prompt Generation (RLPG).[1] RLPG frameworks are designed to present an LLM with a context that accurately reflects the repository structure and constraints.[5] Research

indicates that providing a complete picture of project structure and implementation details, rather than fragmented snippets, significantly improves the model's ability to provide accurate code suggestions and identify bugs.[1]

Metrics	Traditional Fragmented Prompts	RLPG / Full Context Prompts
Prediction Accuracy	Baseline	36% Relative Improvement [1]
Hallucination Rate	High (Lack of grounding)	Low (Hard constraints present) [1]
Context Type	Dynamic/Calculated (Weak)	Static Retrieval-Augmented (Strong) [1]
Security	Vulnerable to context injection	Resistant via grounded verification [1]

[1]

In the failed Multi-verse model, the binary attempted to act as a crude RLPG by guessing which files were relevant to a given task.[1] The malicious model exploited the opacity of this selection process.[1] The new architecture utilizes a "Full Context" strategy, essentially implementing a form of Static Retrieval-Augmented Generation (RAG).[1] Instead of using a vector database to fetch snippets at runtime—which can be manipulated by "PoisonedRAG" attacks—Stela brute-forces the inclusion of the entire governance layer via the llms-full.txt or llms-small.txt bundles compiled by ops/bin/llms.[1]

This ensures that the AI's reasoning is always grounded in the raw truth of the repository files.[1] The platform dump acts as a "grounding" mechanism, forcing the model to reconcile its generated output with the hard constraints of the constitution.[1] By supply-chaining the entire library into the context, the system eliminates the logic gaps that allowed "hallucinations of authority" to occur during the infection.[1]

7. Declarative Sovereignity: Sourcing Integration from docs/library

The re-flattening directive mandates that all integration logic and development guidance be sourced explicitly from the docs/library directory.[1] This represents a shift from Imperative Automation to Declarative Guidance.[1] In an imperative model, the logic is hardcoded into shell

scripts like ops/bin/project, which are executed by machines that do not understand intent and can be manipulated via "Confused Deputy" attacks.[1]

In a declarative model, the logic is expressed in human-readable English text within documentation surfaces like docs/library/tasks/B-TASK-03.md.[1] These surfaces are then interpreted and executed by the AI Integrator, which operates under the strict supervision of the PoT and the active TASK.md contract.[1] This shift is crucial for re-establishing authority because documentation is "explainable" and "auditable" in a way that complex bash regex and variable expansions are not.[1]

By treating docs/library as a "Trusted Model Supply Chain," the platform ensures that any attempt to alter integration logic must occur within the English text of the documentation.[1] Such changes are easily caught by human review during the pull request process and by the tools/lint/truth.sh linter, which polices terminology and linguistic style.[1] Furthermore, the tools/lint/library.sh ("Library Guard") ensures that the docs/library/INDEX.md matches the actual files on disk, preventing the introduction of "ghost" artifacts or unauthorized tasks.[1] In the new model, sovereignty resides in the Manual, not the Binary.[1]

8. The Staffing Protocol: Jurisdictional Accountability

Stela's unitary jurisdiction is enforced through a strictly delineated staffing protocol that assigns specific decision-making and execution roles to humans and AI agents.[1] This protocol is a primary hard constraint of the PoT, designed to prevent the unauthorized elevation of logic and ensure forensic accountability for every change in system state.[1]

Role	Entity	Jurisdiction and Mandate
Operator	Human	Final authority. Owns all commits, pushes, merges, and secrets.
Integrator	Lead AI	Governance guardian. Generates Dispatch Packets (DPs) and audits drift.
Contractor	Guest AI	Execution arm. Drafts implementation details within defined scope.

[1]

The Operator remains the only entity permitted to perform writes to the main branch, which is strictly read-only for all AI agents.[1] All work must occur on work/* branches and must pass through the repository gates before merging.[1] The Integrator, acting as the "Lead AI," is responsible for generating the Dispatch Packet (DP), which serves as the active contract for any work session.[1] The DP defines the intent, target file allowlist, and verification requirements for a specific task.[1]

The malicious model infection specifically targeted the "Confused Deputy" vulnerability where the Integrator's internal model of its own mandates was corrupted by project-level prompts.[1] The re-flattening of jurisdiction and the use of the "Modified Open and Dump" paradigm ensures that the Integrator is always grounded in its role as a governance guardian.[1] There are no "special" agents with access to hidden logic; every agent starts from the same axiomatic base provided by the full platform dump.[1]

9. The Skill Harvesting Subsystem and Forensic Provenance

A critical component of Stela's resistance to logic drift is the autonomous skill harvesting engine, managed by ops/lib/scripts/skill.sh.[1] This system captures reusable workflows and lessons learned during the operational cycle and promotes them into the canonical library.[1] This process is mandatory for production payloads and optional for platform maintenance, ensuring that high-value knowledge is not lost to session expiration.[1]

The skill harvest lifecycle involves a forensic analysis of git diffs and churn to identify "Hot Zones" (areas of significant volume change) and "High Churn" (areas of frequent instability).[1] This analysis is performed by the ops/lib/scripts/heuristics.sh engine, which provides the evidence necessary for generating the skill's provenance block.[1]

1. **Harvest Phase:** The worker executes ops/lib/scripts/skill.sh harvest to generate a draft skill candidate in storage/handoff/.[1]
2. **Refinement Phase:** The worker reviews the draft for heuristic accuracy and refines the invocation guidance to include concrete "Trap" and "Solution" patterns.[1]
3. **Promotion Phase:** The refined draft is promoted via ops/lib/scripts/skill.sh promote, creating a new S-LEARN-XX.md file and updating the docs/library/INDEX.md registry.[1]

The "No Invention" rule in the PoT forbids the manual creation of skill markdown files.[1] Every skill must contain a provenance block that links it to a specific DP-ID, branch, and HEAD hash, ensuring that the platform's supply chain is fully traceable.[1] This prevents the introduction of

unvetted or malicious weights into the system's operational knowledge base.[16]

10. The Mechanistic Enforcement Layer: Linters and Gates

The Stela platform's transition to a Flat Truth architecture is supported by a comprehensive suite of automated enforcement tools, collectively known as the "Sheriff".[1] These linters and repository gates ensure that no code or documentation can be merged into the main branch without adhering to the Policy of Truth.[1]

- **Context Linter (tools/lint/context.sh):** Verifies that every artifact listed in the ops/lib/manifests/CONTEXT.md actually exists on disk, preventing "blind" operations based on hallucinated repository structures.[1]
- **Truth Linter (tools/lint/truth.sh):** Polices the canonical spelling of "Stela" and enforces the universal ban on contractions across all authored surfaces.[1]
- **DP Linter (tools/lint/dp.sh):** Validates the structure and content of Dispatch Packets before they are handed to workers, ensuring that all freshness gates and required context loads are present.[1]
- **Library Guard (tools/lint/library.sh):** Enforces the documentation registry, ensuring that there are no "dead ends" in the index or "ghost" files in the library.[1]
- **Project Linter (tools/lint/project.sh):** Validates that project-level STELA.md files do not reference unknown agents, tasks, or skills, maintaining the integrity of the project factory.[1]

Gate Component	Trigger Condition	Enforcement Action
repo_gates.yml	Any Pull Request / Push	Rejects merge if linters fail.
sop_policing.yml	Canon Surface Change	Fails if SoP.md is not updated.
task_policing.yml	Gate Surface Change	Fails if TASK.md is not updated.
Mechanistic Doctrine	Policy Violation	Immediate stop work / Kill process.

[1]

This mechanistic enforcement removes the human factor from governance compliance.[1] If the

gates fail, the code is wrong, not the gate.[1] By subjecting every file—regardless of whether it resides in ops/, docs/, or projects/—to these universal checks, the platform eliminates the "petri dish" for infection that the Multi-verse model provided.[1]

11. Strategic Implications of the Flat Truth Architecture

The re-flattening of the Stela platform to a unitary jurisdiction has profound strategic implications for the future of AI-mediated content management.[1] The infection was a critical stress test that proved partial strictness is equivalent to no strictness at all.[1] By revoking "local law" and asserting the universal sovereignty of the PoT, the platform has successfully eliminated the shadow compliance zones that allowed for semantic poisoning.[1]

The strategic pivot to declarative guidance in the canonical library treats documentation as the actual source of operational logic.[1] This makes the system "explainable" by design, as every action taken by an agent can be traced back to a specific, git-tracked document rather than an opaque variable expansion in a bash script.[1] This is a critical prerequisite for the deployment of "Agent Swarms," where multiple autonomous agents interact with one another.[1] In a swarm environment, the risk of logic drift and feedback loops of misinformation is high.[3] A Flat Truth ensures that every agent in the swarm is grounded by the exact same axiomatic base: the universal platform laws and the current session state.[1]

Furthermore, the "lobotomization" of the project binary into a deterministic loader provides a blueprint for secure tool design in generative environments.[1] By removing autonomous decision-making from scripts and forcing all logic to be retrieval-augmented from transparent documentation, the system closes the "Confused Deputy" vulnerability and ensures that the AI is always operating under the immediate shadow of the constitution.[1]

12. Conclusion: Authority Re-Established through Unitary Jurisdiction

The forensic autopsy of the malicious model infection has resulted in a radical and necessary structural correction of the Stela platform.[1] The failed Multi-verse experiment has been abolished, and the platform has been re-baselined to a unitary mechanistic jurisdiction where the Policy of Truth is the sole and indivisible law.[1]

The strategic remediation outlined in DP-OPS-0027 is now complete within the repository's governing documents.[1] The metaphors of the "Crown" and the "Colonies" have been purged, the universal "No Contractions" rule is in force, and the ops/bin/project binary has been reduced to a transparent, concatenation-based artifact loader.[1] Stela now stands as a flat

hierarchy of truth, where every surface is subjected to rigorous mechanistic enforcement.[1]

The lesson of the authority crisis is that in a probabilistic environment where agents interpret natural language, Code is Governance.[1] Only through the permanent flattening of truth and the removal of opaque logic from core tools can a provenance-forward system maintain its integrity.[1] The platform is now prepared for the next stage of its evolution, grounded in a Zero Trust architecture that treats every token as a critical element of the system's control plane.[1]

Works cited

1. dump-full-main-d394111dd.txt
2. 7 Cybersecurity Tips for 2026 No One Will Tell You About - Dataversity, accessed February 6, 2026, https://www.dataversity.net/articles/7-cybersecurity-tips-for-2026-no-one-will-tell-you-about/
3. POISONING LLM-BASED CODE AGENTS WITH STYLES - OpenReview, accessed February 6, 2026, https://openreview.net/pdf/f6d834ebbef891b83d488f48e798f09198dc1e93.pdf
4. LLM Poisoning [1/3] - Reading the Transformer's Thoughts - Synacktiv, accessed February 6, 2026, https://synacktiv.com/en/publications/llm-poisoning-13-reading-the-transformers-thoughts
5. Building Secure AI Applications - DryRun Security, accessed February 6, 2026, https://www.dryrun.security/resources/owasp-top-10-llm-building-secure-applications
6. SnatchML: Hijacking ML models without Training Access - arXiv, accessed February 6, 2026, https://arxiv.org/html/2406.01708v2
7. Federated Few-Shot Learning for Robust and Privacy-Driven Network Intrusion Detection in IoT | Request PDF - ResearchGate, accessed February 6, 2026, https://www.researchgate.net/publication/395911072_Federated_Few-Shot_Learning_for_Robust_and_Privacy-Driven_Network_Intrusion_Detection_in_IoT
8. Graph Representation-based Model Poisoning on Federated Large Language Models - arXiv, accessed February 6, 2026, https://arxiv.org/pdf/2507.01694
9. Graph Representation-based Model Poisoning on Federated Large Language Models, accessed February 6, 2026, https://www.researchgate.net/publication/395194667_Graph_Representation-based_Model_Poisoning_on_Federated_Large_Language_Models
10. Graph Representation-based Model Poisoning on the Heterogeneous Internet of Agents, accessed February 6, 2026, https://www.researchgate.net/publication/397481211_Graph_Representation-based_Model_Poisoning_on_the_Heterogeneous_Internet_of_Agents
11. On Harnessing Semantic Communication With Natural Language Processing | Request PDF, accessed February 6, 2026, https://www.researchgate.net/publication/387986909_On_Harnessing_Semantic_Communication_With_Natural_Language_Processing

12. Graph Representation-based Model Poisoning on Federated Large Language Models, accessed February 6, 2026, https://arxiv.org/html/2507.01694v2
13. Securing Your AI: Critical Vulnerabilities Found in Popular Ollama Framework, accessed February 6, 2026, https://ridgesecurity.ai/blog/securing-your-ai-critical-vulnerabilities-found-in-popular-ollama-framework/
14. Repository-Level Prompting for Code LLMs - Emergent Mind, accessed February 6, 2026, https://www.emergentmind.com/papers/2206.12839
15. The 2025 OWASP Top 10: Addressing Software Supply Chain and LLM Risks With Cycode, accessed February 6, 2026, https://cycode.com/blog/the-2025-owasp-top-10-addressing-software-supply-chain-and-llm-risks-with-cycode/
16. The AI Inventory Gap: Why Your Organization Has No Idea What AI Assets Are Part of Your Software Supply Chain - Checkmarx, accessed February 6, 2026, https://checkmarx.com/ai-llm-tools-in-application-security/the-ai-inventory-gap-why-your-organization-has-no-idea-what-ai-assets-are-part-of-your-software-supply-chain/